EXPERIMENT-03 :

Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

CODE

```
# Decision Tree ID3 Algorithm

# Dataset + Entropy + Information Gain + Classification + Matplotlib


import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.preprocessing import LabelEncoder


# 1. Create the dataset

data = {

   'Outlook': [

      'Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain',

      'Rain', 'Overcast', 'Sunny', 'Sunny', 'Rain',

      'Sunny', 'Overcast', 'Overcast', 'Rain'

   ],

   'Temperature': [

      'Hot', 'Hot', 'Hot', 'Mild', 'Cool',

      'Cool', 'Cool', 'Mild', 'Cool', 'Mild',

      'Mild', 'Mild', 'Hot', 'Mild'

   ],

   'Humidity': [
```

```
        'High', 'High', 'High', 'High', 'Normal',
        'Normal', 'Normal', 'High', 'Normal', 'Normal',
        'Normal', 'High', 'Normal', 'High'
    ],
    'Wind': [
        'Weak', 'Strong', 'Weak', 'Weak', 'Weak',
        'Strong', 'Strong', 'Weak', 'Weak', 'Weak',
        'Strong', 'Strong', 'Weak', 'Strong'
    ],
    'PlayTennis': [
        'No', 'No', 'Yes', 'Yes', 'Yes',
        'No', 'Yes', 'No', 'Yes', 'Yes',
        'Yes', 'Yes', 'Yes', 'No'
    ]
}

df = pd.DataFrame(data)

print("TRAINING DATASET")
print(df.to_string(index=False))

# 2. Calculate entropy
def entropy(labels):
    probabilities = labels.value_counts(normalize=True)
    return -sum(p * np.log2(p) for p in probabilities if p > 0)
```

```
# 3. Calculate information gain
def information_gain(df, feature, target):
    total_entropy = entropy(df[target])
    weighted_entropy = 0

    for value in df[feature].unique():
        subset = df[df[feature] == value]
        weighted_entropy += (
            len(subset) / len(df)
        ) * entropy(subset[target])

    return total_entropy - weighted_entropy

target = 'PlayTennis'
features = list(df.columns[:-1])

print("\nINFORMATION GAIN")
for feature in features:
    print(f"{feature}: {information_gain(df, feature, target):.4f}")

# 4. Encode categorical features
encoders = {}
X = pd.DataFrame()

for column in features:
    encoders[column] = LabelEncoder()
```

```
    X[column] = encoders[column].fit_transform(df[column])

y_encoder = LabelEncoder()
y = y_encoder.fit_transform(df[target])

# 5. Build ID3 decision tree
# Entropy criterion uses information gain.
model = DecisionTreeClassifier(
    criterion='entropy',
    random_state=0
)
model.fit(X, y)

# 6. Display decision tree using Matplotlib
plt.figure(figsize=(16, 8))
plot_tree(
    model,
    feature_names=features,
    class_names=list(y_encoder.classes_),
    filled=True,
    rounded=True,
    fontsize=10
)
plt.title("Decision Tree using ID3 (Information Gain)")
plt.show()
```

```
# 7. Classify a new sample
new_sample = {
    'Outlook': 'Sunny',
    'Temperature': 'Cool',
    'Humidity': 'High',
    'Wind': 'Strong'
}

new_df = pd.DataFrame([new_sample])

for column in features:
    new_df[column] = encoders[column].transform(new_df[column])

prediction = model.predict(new_df[features])
result = y_encoder.inverse_transform(prediction)[0]

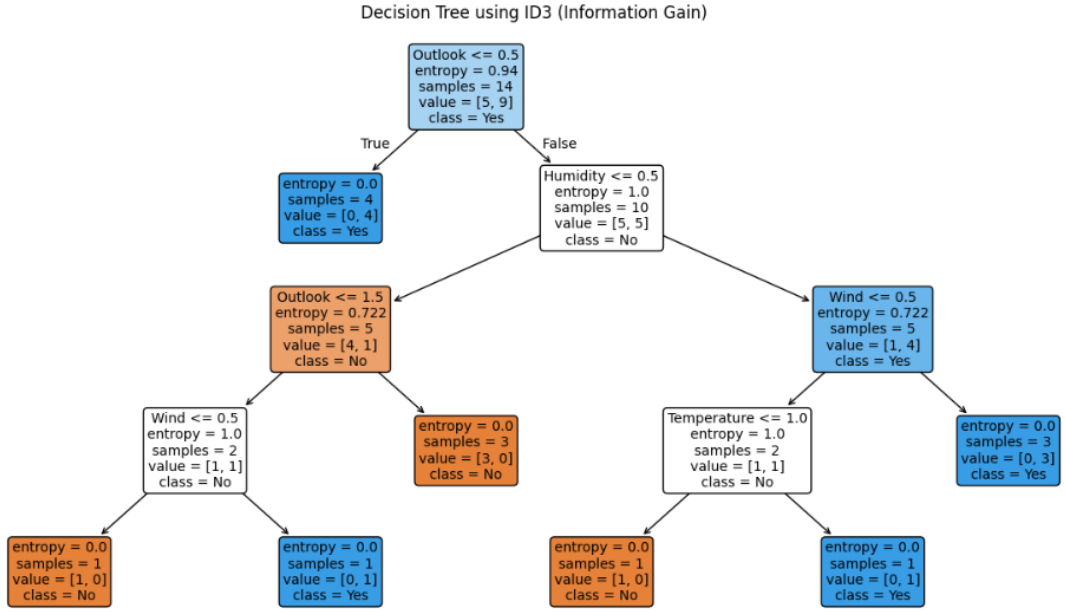
print("\nNEW SAMPLE")
print(new_sample)

print("\nCLASSIFICATION RESULT")
print("Play Tennis:", result)
```

OUTPUT:

```
      Sunny        Mild   Normal Strong        Yes
Overcast        Mild     High Strong        Yes
Overcast         Hot   Normal   Weak        Yes
    Rain        Mild     High Strong         No

INFORMATION GAIN
Outlook: 0.2467
Temperature: 0.0292
Humidity: 0.1518
Wind: 0.0481
```




```
NEW SAMPLE
```